

Chapter 7 — Neural networks for PDEs

Linear versus nonlinear approximation: what neural networks buy, and what they cost

MTH3340 · Numerical Methods for PDEs

1 The structure of the methods in this course

All the methods of this course follow the same sequence:

$\text{PDE} \rightarrow \text{weak form} \rightarrow \text{Galerkin on } V_h \rightarrow \text{error analysis} \rightarrow Au = f.$

Every method we met (spectral, finite elements; also finite differences and finite volumes) shares one structural feature:

the discrete space V_h is a vector space, chosen *before* we know anything about u .

This is called **linear approximation**. The payoff was a *convex* problem with a unique solution, computable by solving a linear system (Chapters 1–2, 6).

This chapter: the limits of this choice, and a family of **nonlinear** approximation methods based on neural networks, analysed with the tools developed in the previous chapters.

2 The price of linear approximation

Céa's lemma + interpolation estimates (Chapter 2), finite elements of order p in d dimensions:

$$|u - u_h|_{H^1(\Omega)} \leq C h^p |u|_{H^{p+1}(\Omega)}.$$

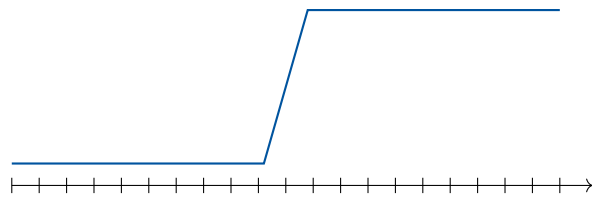
To reach an accuracy ε on a quasi-uniform mesh we need $h \sim \varepsilon^{1/p}$, i.e.

$$N \sim h^{-d} \sim \varepsilon^{-d/p} \quad \text{degrees of freedom} \quad (\text{notes, (7.1)}).$$

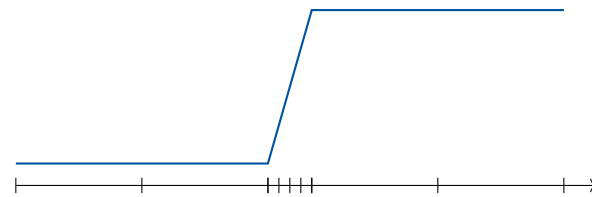
- exponential in the dimension: the **curse of dimensionality**;
- harmless for $d = 1, 2, 3$ and moderate accuracies — the regime where finite elements excel;
- prohibitive for parametric or phase-space problems ($d \gg 3$).

3 Multiscale solutions

Linear approximation is also wasteful in *low* dimension when the solution has localised features: the method cannot exploit them. A solution that is constant except across a layer of width $\delta \ll 1$ (Chapter 4):



uniform, $N \sim 1/\delta$



adapted, $N = O(1)$

A uniform mesh needs $h \lesssim \delta$ *everywhere*; most cells are wasted where nothing happens. A mesh refined only inside the layer does the job with N independent of δ .

The difficulty: the position of the layer must be known in advance. The mesh depends on the solution — **this is no longer linear approximation.**

4 Adaptive finite elements

The classical escape route. The key tool is an **a posteriori error estimator** $\eta_K(u_h)$, computable from u_h on every cell. For $-\Delta u = f$ with linear elements,

$$\eta_K(u_h)^2 \doteq h_K^2 \|f + \Delta u_h\|_{L^2(K)}^2 + \frac{1}{2} \sum_{F \subset \partial K \setminus \partial \Omega} h_F \|[\nabla u_h \cdot \mathbf{n}]\|_{L^2(F)}^2,$$

cell residual plus **jumps of the normal flux**. It is **reliable** and (locally) **efficient**:

$$|u - u_h|_{H^1(\Omega)} \leq C_{\text{rel}} \left(\sum_K \eta_K(u_h)^2 \right)^{1/2} \quad (\text{reliable}),$$

$$\eta_K(u_h) \leq C_{\text{eff}} |u - u_h|_{H^1(\omega_K)} + \text{data oscillation} \quad (\text{efficient}),$$

ω_K the patch of cells around K . Not proved in this unit; see the notes, §7.1.1.

5 The adaptive algorithm

Given an estimator, adaptivity is a loop over meshes: solve, estimate the error cell by cell, refine where it is largest.

Algorithm (Adaptive finite element method — notes, Algorithm 7.1)

```
for  $\alpha = 0, 1, \dots$ , till the estimated error is below the tolerance do  
    SOLVE the finite element problem on  $\mathcal{T}_\alpha$  for  $u_\alpha \in V_\alpha$   
    ESTIMATE the error, computing  $\eta_K(u_\alpha)$  for all  $K \in \mathcal{T}_\alpha$   
    MARK the cells with the largest  $\eta_K$   
    REFINES the marked cells to obtain  $\mathcal{T}_{\alpha+1}$   
end for
```

Reliability makes the estimated error trustworthy as a stopping criterion; efficiency guarantees that we do not refine where the true error is small. The estimator is **problem-dependent**: a new one must be designed, and proved reliable and efficient, for each class of problems.

6 What adaptivity buys

The spaces $V_0, V_1, \dots$ *do* depend on the solution. Each V_α is still a vector space: **adaptivity is nonlinear approximation built out of linear pieces**. The price is a **two-loop** algorithm: meshes outside, a full finite element solve inside.

A 2D domain with a re-entrant corner, $u \sim r^\lambda$ near it ($\lambda < 1$), smooth elsewhere:

$$|u - u_N|_{H^1(\Omega)} \leq C \begin{cases} N^{-\lambda/2} & \text{uniform refinement, any order } p, \\ N^{-p/2} & \text{adaptive refinement, order } p. \end{cases}$$

- on uniform meshes the rate is capped by the **regularity**, not by the method: raising p buys nothing;
- adaptivity recovers the rate of the smooth case: meshes graded towards the corner compensate the singularity.

7 Kolmogorov widths (advanced, §7.1.2)

A bump η of width δ travelling at unit speed solves $\partial_t u + \partial_x u = 0$. One space V_n for *all* times must resolve the family $\mathcal{K} \doteq \{\eta(\cdot - t) : t \in [0, 1]\}$: it needs $n \sim 1/\delta$ basis functions, although each snapshot alone needs $\mathcal{O}(1)$ on an adapted mesh.

Definition (Kolmogorov n -width)

$$d_n(\mathcal{K}) \doteq \inf_{\substack{V_n \subset V \\ \dim V_n = n}} \sup_{u \in \mathcal{K}} \inf_{v_n \in V_n} \|u - v_n\|,$$

the best worst-case error of *any* n -dimensional linear space.

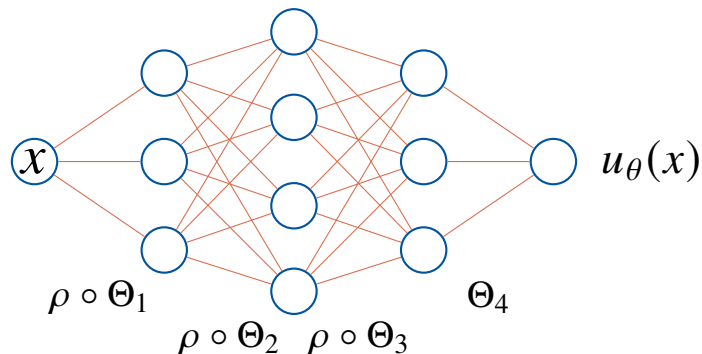
No linear method can beat $d_n(\mathcal{K})$ on $\mathcal{K}$. For transport-dominated families it decays very slowly in n : the **Kolmogorov barrier**. Nonlinear methods are not bound by it.

8 Neural networks

Definition (Fully-connected feed-forward neural network — notes, Def. 7.2.1)

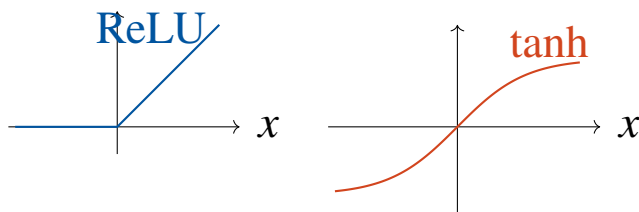
Architecture $(n_0, \dots, n_L)$ with $n_0 = d$, $n_L = 1$; affine maps $\Theta_k(\mathbf{x}) \doteq W_k \mathbf{x} + \mathbf{b}_k$ with $W_k \in \mathbb{R}^{n_k \times n_{k-1}}$, $\mathbf{b}_k \in \mathbb{R}^{n_k}$; an activation $\rho : \mathbb{R} \rightarrow \mathbb{R}$ applied component-wise. The realisation with parameters $\theta = \{W_k, \mathbf{b}_k\}_{k=1}^L$ is

$$u_\theta \doteq \Theta_L \circ \rho \circ \Theta_{L-1} \circ \dots \circ \rho \circ \Theta_1.$$



Architecture $(1, 3, 4, 3, 1)$: layers $1, \dots, L - 1$ are **hidden**, n_k = neurons of layer k . Each edge carries a weight, each neuron adds a bias and applies ρ (not in the last layer). Parameters: $P = \sum_k n_k(n_{k-1} + 1)$.

9 Activations, and a family that is not a space



$$\text{ReLU}(x) = \max(0, x), \quad \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

The regularity of u_θ is that of ρ :
ReLU networks are continuous piecewise linear, tanh networks are C^∞ .

The set of all realisations of a fixed architecture,

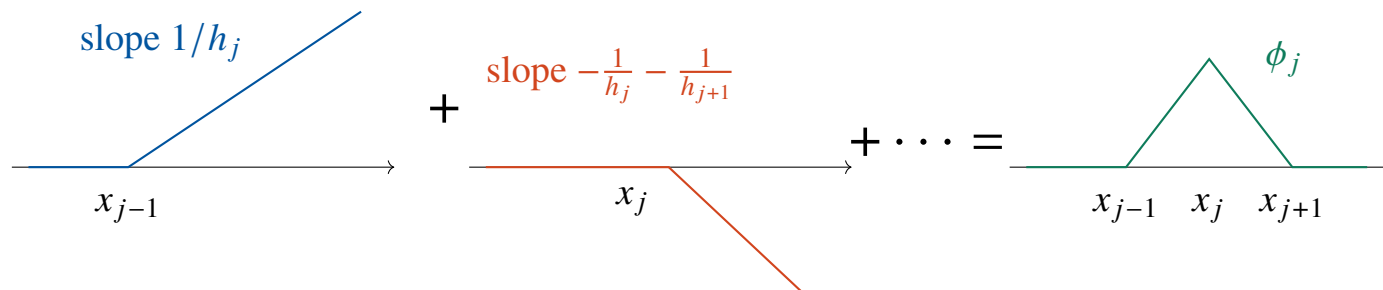
$$\mathcal{N} \doteq \{u_\theta : \theta \in \mathbb{R}^P\} \quad (\text{notes, (7.2)}),$$

is **not a vector space**: the hidden parameters enter nonlinearly, through compositions with ρ ; only the last layer is linear. The sum of two realisations is in general not a realisation (Tutorial). **Approximating in $\mathcal{N}$ is nonlinear approximation**: the parameters can adapt to the function, like the adapted mesh of board 3.

10 A hat function out of three ReLUs

Nodes $x_{j-1} < x_j < x_{j+1}$, cell sizes h_j, h_{j+1} . For every $x \in \mathbb{R}$ (notes, Example 7.2.2),

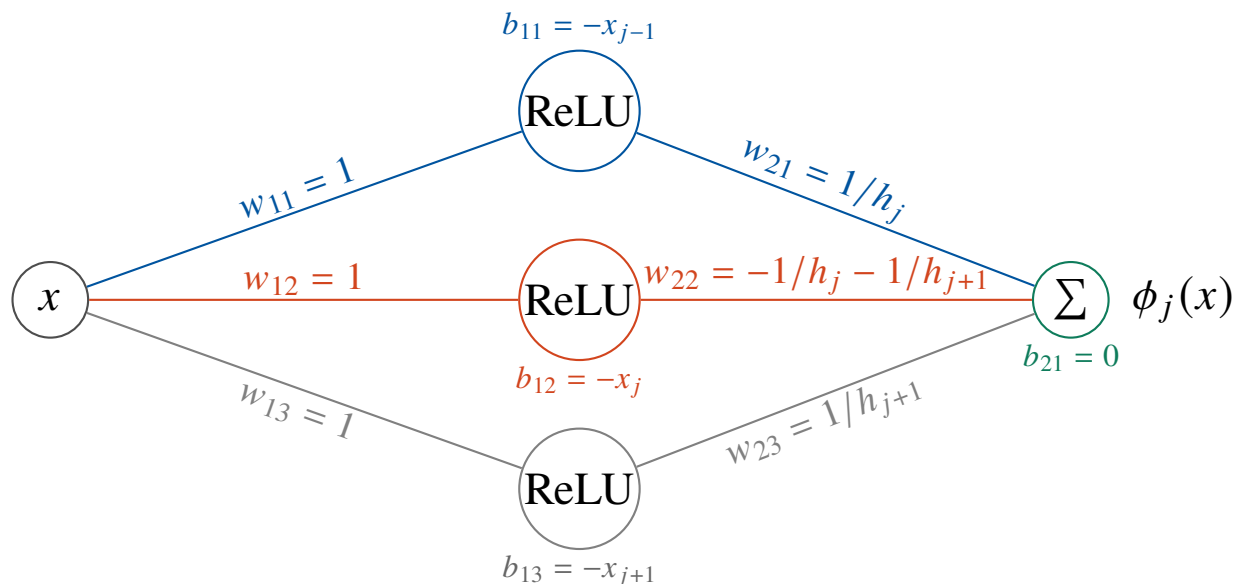
$$\phi_j(x) = \frac{1}{h_j} \text{ReLU}(x - x_{j-1}) - \left(\frac{1}{h_j} + \frac{1}{h_{j+1}} \right) \text{ReLU}(x - x_j) + \frac{1}{h_{j+1}} \text{ReLU}(x - x_{j+1}).$$



Check the slopes cell by cell: 0, then $1/h_j$, then $1/h_j - 1/h_j - 1/h_{j+1}$, then 0.

This is a network of architecture $(1, 3, 1)$: hidden weights 1, hidden biases $-x_{j-1}, -x_j, -x_{j+1}$, output weights the three slopes (next board). **Hat functions are realisations of small neural networks.**

11 The hat function as a (1, 3, 1) network



Weights on the edges, **one bias per neuron** (w_{ki} , b_{ki} : entries of W_k , $\mathbf{b}_k$); hidden neuron i outputs $\text{ReLU}(w_{1i}x + b_{1i})$, a ridge with kink at $-b_{1i}$.

- moving the **biases** moves the kinks, i.e. the **mesh nodes**;
- changing the **output weights** changes the slopes;
- replacing ReLU by a smooth activation rounds off the kinks.

12 Neural networks contain finite elements

Theorem (Finite element emulation in 1D — notes, Theorem 7.2.3)

Let V_h be the linear finite element space on a mesh of $[a, b]$ with M cells and nodes $x_0 < \dots < x_M$, and $\mathcal{N}$ the realisations of $(1, n_1, 1)$ with activation ReLU, $n_1 \geq M$. Then $V_h \subset \mathcal{N}$ on $[a, b]$.

Let s_j be the slope of v_h on (x_{j-1}, x_j) . For every $x \in [a, b]$,

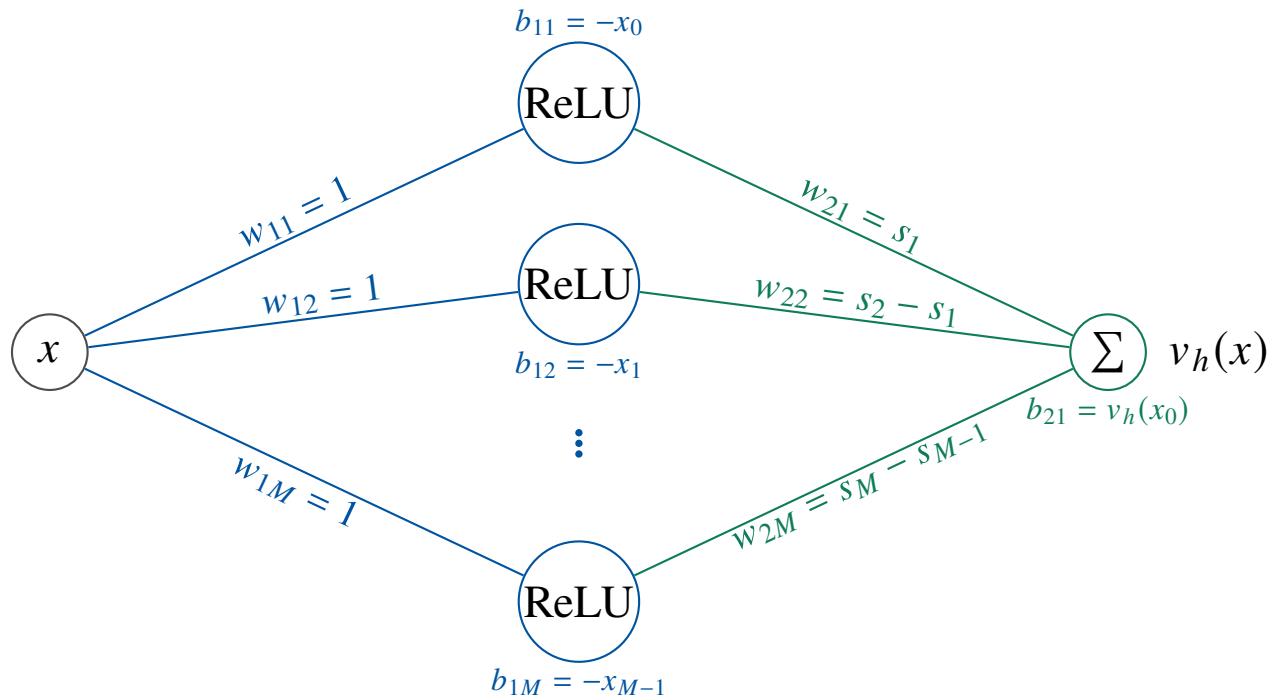
$$v_h(x) = v_h(x_0) + s_1 \text{ReLU}(x - x_0) + \sum_{j=1}^{M-1} (s_{j+1} - s_j) \text{ReLU}(x - x_j) \quad (\text{notes, (7.4)}).$$

Both sides are continuous piecewise linear with the same breakpoints and the same value at x_0 ; on (x_{k-1}, x_k) the right-hand side has slope $s_1 + \sum_{j < k} (s_{j+1} - s_j) = s_k$. Reading the parameters in two parts, with $s_0 \doteq 0$:

- the constant $v_h(x_0)$ is the bias $b_2 \in \mathbb{R}^{n_2} = \mathbb{R}$ of the last layer: biases are *per neuron* (Def. 7.2.1, $\mathbf{b}_k \in \mathbb{R}^{n_k}$), and the last layer has a single neuron;
- neurons $j = 1, \dots, M$, one per cell: first-layer weight 1 and bias $-x_{j-1}$, so neuron j outputs $\text{ReLU}(x - x_{j-1})$; last-layer weight $s_j - s_{j-1}$, the slope jump.

Spare neurons get output weight 0. □

13 The emulating $(1, M, 1)$ network



Neuron j switches on at the node x_{j-1} and adds the slope jump $s_j - s_{j-1}$ ($s_0 \doteq 0$); the output bias sets the value at x_0 . All the **nodes** are **biases**: trainable.

14 What this means for approximation

Since $V_h \subset \mathcal{N}$,

$$\inf_{v \in \mathcal{N}} \|u - v\| \leq \inf_{v_h \in V_h} \|u - v_h\| \leq C h |u|_{H^2(\Omega)} \quad (\text{notes, (7.5)}).$$

Networks are **at least as expressive** as the finite element spaces they emulate.

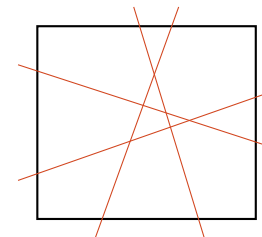
But this is a *linear* bound on a *nonlinear* family, far from sharp:

- the breakpoints are the hidden biases — **trainable**. A one-hidden-layer ReLU network in 1D is a continuous piecewise linear function with n_1 **free knots**;
- the same architecture contains V_h on *every* mesh with M cells, so the bound holds for the best *adaptive* finite element approximation with M cells;
- training moves the mesh: **mesh adaptation without an estimator and without the two-loop algorithm** of board 4.

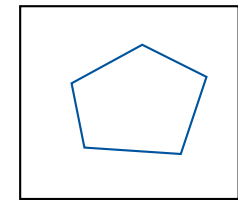
15 Higher dimensions: the need for depth

One hidden layer = sum of ridge functions $\rho(\mathbf{w} \cdot \mathbf{x} + b)$: functions of the single variable $\mathbf{w} \cdot \mathbf{x} + b$. For ReLU, one kink (crease) along the full hyperplane $\mathbf{w} \cdot \mathbf{x} + b = 0$.

- a 2D hat function has creases along bounded edges;
- sums of full creases cannot stop at a vertex: one hidden layer cannot represent a 2D hat function, whatever its width;
- two hidden layers: the second cuts the creases of the first.



one hidden layer



two hidden layers

Known: continuous piecewise linear on $\mathbb{R}^d \Rightarrow$ ReLU network with $\lceil \log_2(d + 1) \rceil$ hidden layers; linear finite elements on 2D/3D simplicial meshes: two. The emulation bound survives, at the price of depth.

16 Universal approximation

Theorem (Universal approximation — notes, Theorem 7.2.4)

Let $\rho \in C^0(\mathbb{R})$ and $K \subset \mathbb{R}^d$ compact. One-hidden-layer realisations with activation ρ and arbitrary width are dense in $C^0(K)$ (maximum norm) if and only if ρ is not a polynomial.

Reassuring, but **much weaker than it looks**: a density statement, like polynomials in C^0 .

- nothing about **rates**: how many neurons for a given accuracy?
- nothing about **how to find the parameters**.

Finite elements give both a rate and an algorithm (solve a linear system) that attains it. For networks the emulation results give (non-optimal) rates; the algorithmic question is **training**, where the real difficulties live.

17 Training

The native task: **data fitting**. Samples $(\mathbf{x}_i, y_i)$, $i = 1, \dots, N_s$, a **loss** and a minimisation problem,

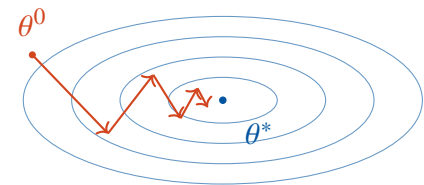
$$\mathcal{L}(\theta) \doteq \frac{1}{N_s} \sum_{i=1}^{N_s} |u_{\theta}(\mathbf{x}_i) - y_i|^2, \quad \theta^* \in \arg \min_{\theta \in \mathbb{R}^P} \mathcal{L}(\theta).$$

Algorithm (Gradient descent — notes, Algorithm 7.2)

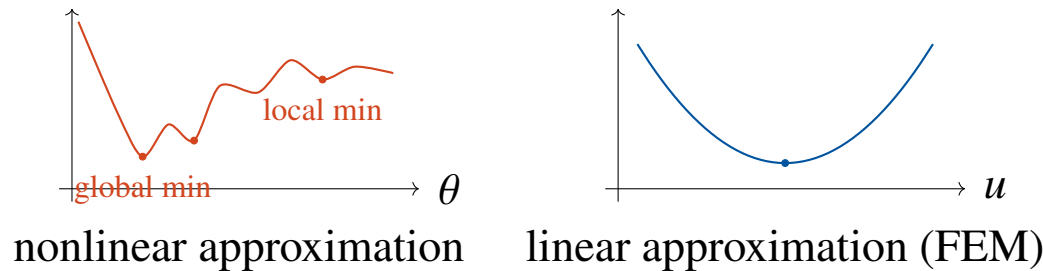
random θ^0 , learning rate $\alpha > 0$; **for** $k = 0, 1, \dots$, till convergence:
 $\mathbf{g}^k := \nabla_{\theta} \mathcal{L}(\theta^k), \quad \theta^{k+1} := \theta^k - \alpha \mathbf{g}^k$

The nonlinear Richardson iteration of Chapter 6; the **learning rate** is the relaxation parameter. In practice: Adam (adaptive steps, momentum), stochastic gradients.

All gradients — and later $\partial_{\mathbf{x}} u_{\theta}$ — by **automatic differentiation**: the chain rule through Def. 7.2.1, exact to machine precision.



18 Non-convexity of the training problem



Galerkin for a coercive problem: the discrete energy $\frac{1}{2}u^T A u - u^T f$ is **convex** in the DOFs (A s.p.d.), one global minimum, one linear solve.

The loss $\mathcal{L}(\theta)$ is **non-convex**: the parameters enter nonlinearly. Local minima, saddles, flat regions; the result depends on the random initialisation; **no convergence guarantee to the global minimum**.

This is the price of nonlinear approximation: the approximation power of the family and the computability of the minimiser are in tension.

19 Physics-informed neural networks

Abstract boundary value problem of Chapter 1: $\mathcal{L}u = f$ in Ω , $u = u_D$ on Γ_D , $\mathbf{n} \cdot \mathcal{F}(u) = g_N$ on Γ_N . The idea is simple: take $u_\theta \in \mathcal{N}$ and minimise the **strong residual**, boundary conditions by penalty,

$$\mathcal{L}(\theta) \doteq \int_{\Omega} (\mathcal{L}u_\theta - f)^2 + \beta_D \int_{\Gamma_D} (u_\theta - u_D)^2 + \beta_N \int_{\Gamma_N} (\mathbf{n} \cdot \mathcal{F}(u_\theta) - g_N)^2$$

(notes, (7.8)). A classical solution is a zero, hence a global minimiser, of this non-negative functional.

Nothing more can be claimed.

- residuals measured in L^2 -type norms, *not* the norms of well-posedness (H^1 for u , H^{-1} for f , $H^{1/2}(\Gamma_D)$ for u_D): no stability, no theory behind it;
- needs $\mathcal{L}u_\theta \in L^2(\Omega)$: more regularity than the weak setting (fine for smooth activations).

20 Monte Carlo and the discrete loss

Definition (Monte Carlo integration — notes, Def. 7.4.1)

For $\{\mathbf{p}_i\}_{i=1}^N$ independent, uniform random points in Ω ,

$$\int_{\Omega} g \approx \frac{|\Omega|}{N} \sum_{i=1}^N g(\mathbf{p}_i), \quad \text{root mean square error } |\Omega| \sigma(g) N^{-1/2},$$

$\sigma(g)^2$ the variance of g over Ω . The rate is independent of d — and very slow.

The PINN method replaces the integrals by averages at random **collocation points** $\{\mathbf{p}_i\} \subset \Omega$, $\{\mathbf{q}_i\} \subset \partial\Omega$,

$$\mathcal{L}_N(\theta) \doteq \frac{1}{N_p} \sum_i |\mathcal{L}u_{\theta}(\mathbf{p}_i) - f(\mathbf{p}_i)|^2 + \frac{\gamma}{N_{\partial}} \sum_i |u_{\theta}(\mathbf{q}_i) - u_D(\mathbf{q}_i)|^2,$$

(notes, (7.9)), derivatives by automatic differentiation. **Mesh-free, a few lines of code, nonlinear operators verbatim**: hence its popularity. The idea dates from the 1990s.

21 The integration problem

The discrete loss only *feels* the residual at the sampling points.

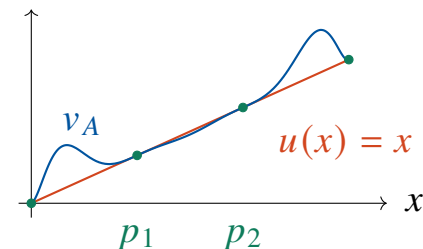
Example (Zero loss, wrong solution — notes, Ex. 7.4.2)

$-u'' = 0$ on $(0, 1)$, $u(0) = 0$, $u(1) = 1$, so $u(x) = x$. For fixed collocation points $p_1, \dots, p_{N_p}$ and any amplitude $A \in \mathbb{R}$,

$$v_A(x) \doteq x + A x^2(1-x)^2 \prod_{i=1}^{N_p} (x - p_i)^3$$

has exact boundary values and $v_A''(p_i) = 0$: $\mathcal{L}_N(v_A) = 0$ for every A .

An expressive network can approximate any v_A : the discrete loss has **near-zero-loss configurations arbitrarily far from the solution**; the continuous loss would reject them. Remedies (more points, resampling, adaptive points) help, but restore **no guarantee**.



22 The boundary condition problem

Finite elements impose Dirichlet conditions **strongly**, by construction of the space. On $\mathcal{N}$ there is no such handle; the penalty is a weak surrogate with a trade-off:

small β_D : boundary violated

large β_D : stiff loss, slow training.

Deeper: the datum lives in $H^{1/2}(\Gamma_D)$; penalising in $L^2(\Gamma_D)$ is a **norm mismatch** that pollutes the approximation.

Exact imposition by construction on $\Omega = [0, 1]^d$ with $u = 0$ on $\partial\Omega$:

$$\xi(\mathbf{x}) \doteq \prod_{i=1}^d x_i(1 - x_i), \quad \tilde{u}_\theta \doteq \xi u_\theta,$$

vanishes on $\partial\Omega$ for every θ ; the boundary terms disappear and training improves markedly (non-homogeneous data: add a lifting, as with the offset function).

Weakness: geometry. On a general domain no explicit smooth function vanishes exactly on $\partial\Omega$ and only there.

23 Assessment for forward low-dimensional problems

How do PINNs compare with the methods of this course on forward, low-dimensional PDEs?

- the literature reports many successes; few survive a **comparison against a properly configured standard solver**;
- an experience report on a 2D flow benchmark: **about 32 hours** of PINN training to match what a standard solver gave in **about 20 seconds**; the physics (vortex shedding) missed on a second benchmark.

No contradiction with the theory: Galerkin solves a **convex** problem with guaranteed error and optimal complexity; a PINN attacks a **non-convex** problem for the same solution, with inexact integration, penalised boundary conditions and no error control.

24 Variational losses: deep Ritz

The strong residual is only one choice. For a symmetric coercive problem, Chapter 1 characterises u as the global minimiser of $J(v) = \frac{1}{2}a(v, v) - \ell(v)$ over V . The **deep Ritz method** minimises the *same* energy over the family:

$$\min_{\theta \in \mathbb{R}^P} J(u_\theta) + \beta_D \int_{\Gamma_D} (u_\theta - u_D)^2.$$

The quadratic minimisation problem of Def. 1.2.2 posed on $\mathcal{N}$ instead of a vector space; only first derivatives, so milder regularity.

Three warnings:

1. J restricted to $\mathcal{N}$ is **not convex** in θ ;
2. **existence is not guaranteed**: $\mathcal{N}$ is not closed, minimising sequences can degenerate (weights blow up) — the non-existence example of Chapter 1 revisited;
3. the energy is not signed: its minimum is negative and unknown, so the loss value is **no error indicator**.

25 A one-parameter deep Ritz landscape

Poisson problem $u'' = f''$ on $\Omega = (-1, 1)$ with $u = f$ at $x = \pm 1$, manufactured solution $f = \text{ReLU}$, and the [one-neuron, one-parameter model](#)

$$u_c(x) \doteq \text{ReLU}(x + c), \quad c \in \mathbb{R} \quad (\text{exact solution: } c = 0).$$

Loss: the [deep Ritz energy](#) of the previous board, i.e. the energy of Chapter 1 with the Dirichlet conditions imposed by a penalty; for $-v'' = \tilde{f}$, $v(\pm 1) = g(\pm 1)$,

$$\mathcal{J}(v) \doteq \underbrace{\frac{1}{2} \int_{-1}^1 (v')^2 - \int_{-1}^1 \tilde{f} v}_{\text{energy of Chapter 1}} + \underbrace{\frac{\beta}{2} \sum_{b=\pm 1} (v(b) - g(b))^2}_{\text{penalty, } \beta > 0}.$$

Bounded from below and convex in v , like the energies of Chapter 1. For our data f'' is a Dirac delta at 0, so $-\int \tilde{f} v = v(0)$, and $\mathcal{J}(u_c)$ is a function of c alone.

26 The energy as a function of the parameter

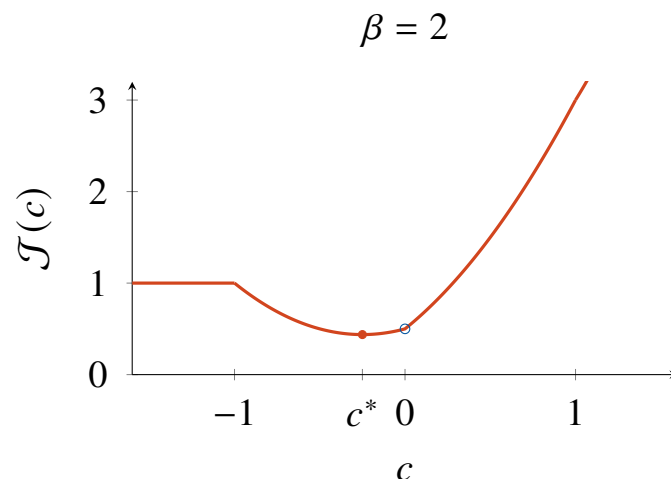
Closed form (notes, §7.3.1):

$$\mathcal{J}(c) = \begin{cases} \frac{\beta}{2} & c < -1, \\ \frac{1}{2}(\beta c^2 + c + 1) + \max(c, 0) & -1 \leq c \leq 1, \\ 1 + c + \frac{\beta}{2}((c-1)^2 + c^2) & c > 1. \end{cases}$$

Parabola on $[-1, 0]$, increasing on $[0, 1]$:
the global minimiser is

$$c^* = -\frac{1}{2\beta} \neq 0.$$

Note the **flat region**, the **kink at $c = 0$** , and where the minimum is.



27 Four diseases, and a cure for one

Already with *one* parameter:

1. **a flat region**: for $c < -1$ the kink leaves the domain, $u_c \equiv 0$ on Ω , the gradient vanishes — an initialisation $c < -1$ never recovers;
2. **inconsistency**: the minimiser is $c^* = -1/(2\beta)$, not the exact solution; the error is $O(1/\beta)$ and $c = 0$ is stationary only as $\beta \rightarrow \infty$;
3. **stiffness**: to reduce the consistency error we want $\beta \rightarrow \infty$, but the curvature at the minimum is β , so gradient descent converges only for $\alpha\beta < 2$: smaller steps and more iterations (the boundary-condition trade-off, board 22);
4. **a gradient jump** at $c = 0$ (ReLU not differentiable at 0). Harmless here; with a consistent loss the minimum sits at the kink and gradient descent hops around it forever (Magueresse–Badia, 2024).

Cure for 4: regularity — a C^2 activation gives a continuous loss gradient; hence **smooth activations** (tanh and relatives) for PDEs.

28 Summary

- This course: **linear approximation**, spaces chosen a priori — efficient in low dimension, limited for multiscale solutions, unaffordable in high dimension.
- Neural networks: **nonlinear parametrisations** that *contain* finite element spaces and adapt to the solution, playing the role of an adaptive mesh.
- The price: **non-convex training**, **approximate integration** with spurious minimisers, **no strong boundary conditions**, no error control.
- Forward low-dimensional problems: **not competitive** with finite elements.

29 Where does this leave us?

- The field is not new: the current wave is a decade old, the idea dates from the 1990s. After a very large literature, **these methods are not used in engineering practice** for forward PDE approximation.
- Neural networks excel in **data-rich** problems (weather prediction, transformative); they struggle in the **data-scarce** regimes where PDE models matter most (climate projection).
- Will it change? Perhaps — but likely not with ever-larger ML-style architectures: **nonlinear approximations designed for PDEs from the start**, with better integration, better-posed minimisation, interpretable architectures (e.g. free-knot splines).

The analysis of this course provides the tools to assess this literature.